

TP4 – Pilotage de LED et mémoire Partie 2

Questions 1. Sur l'architecture RTL, modifiez le module LED_driver en ajoutant une sortie end_cycle. Cette sortie vaudra 1 à la fin d'un cycle allumé/éteint de la LED RGB.

```
end_cycle_process : process(clk, resetn)
begin
    if resetn = '1' then

        end_cycle_s <= '0';

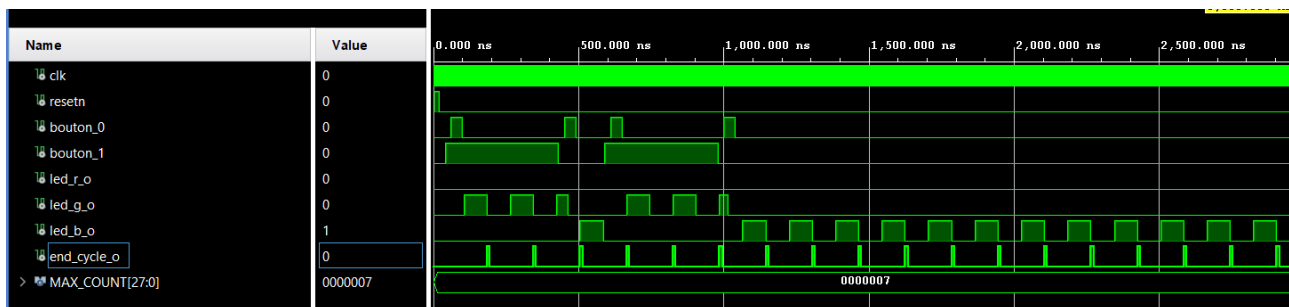
    elsif rising_edge(clk) then

        end_cycle_s <= '0'; -- valeur par défaut

        if end_counter_s = '1' and cpt(0) = '1' then

            end_cycle_s <= '1';

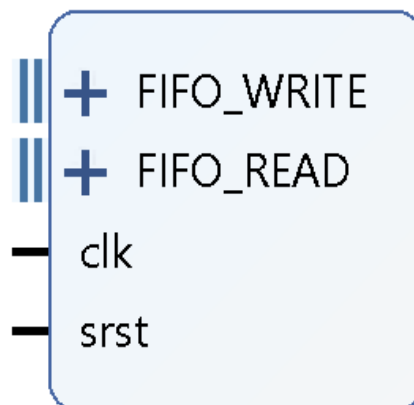
        end if;
    end if;
end process end_cycle_process;
```



Nous avons bien end_cycle qui passe à '1' dès lors qu'un cycle ON/OFF d'une LED est effectué.

2. Modifiez la logique en entrée du module pour ajouter une FIFO. Cette FIFO doit prendre en entrée le code couleur « vert » ou « bleu » suivant l'état du bouton_1 et est connectée en sortie à l'entrée color_code du module LED_driver. La donnée est écrite dans la FIFO lorsqu'il y a un front montant du bouton_0. La donnée de la FIFO est lue lorsque le signal end_cycle du module LED_driver vaut 1. Pour ajouter une FIFO, ouvrez le catalog d'IP de Vivado (onglet Project Manager -> IP Catalog) et recherchez l'IP qui vous semble la plus pertinente. Les documentations des IPs de Xilinx sont disponibles en ligne, cela peut faciliter vos recherches. Une fois que vous avez choisi une IP dans le catalogue, double cliquez dessus et choisissez les paramètres. Cliquez ensuite sur OK. Pour vous aider pour l'implémentation de cette IP, vous pouvez ouvrir un design d'exemple. Pour cela, faites un clic droit sur l'IP dans l'onglet Sources, puis Open IP Example Design.

J'ai donc chosi l'IP FIFO Generator:



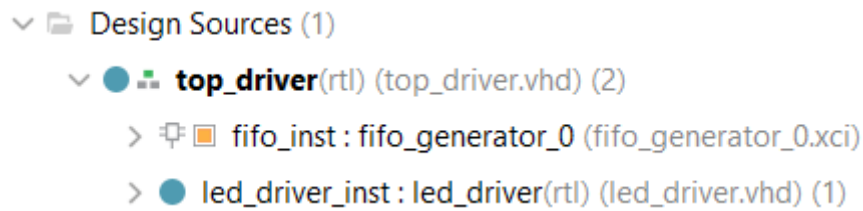
Je l'ai ensuite intégré dans le top en le connectant à des signaux internes intermédiaires:

```
-----
-- Instance de la FIFO
-----

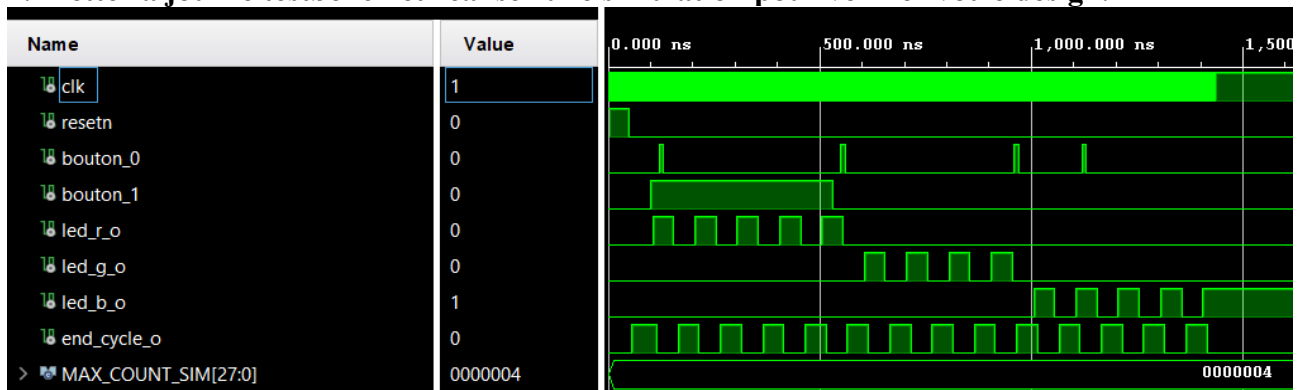
fifo_inst : fifo_generator_0
  PORT MAP (
    clk    => clk,
    srst   => resetn,
    din    => color_in_s,
    wr_en  => update_s,
    rd_en  => end_cycle_s,
    dout   => color_out_s,
    full   => fifo_full_s,
    empty  => fifo_empty_s
  );
```

3. Modifiez vos codes de la partie 1 pour y ajouter les nouveaux éléments de votre architecture.

J'ai donc designé un top, englobant la FIFO et mon module led_driver:



4. Mettez à jour le testbench et réalisez une simulation pour vérifier votre design.



-resetn fait bien son impulsion initiale à '1' puis repasse à '0' ce qui est conforme au stimulus du testbench.

-bouton_0 génère bien des impulsions brèves d'un cycle, à plusieurs moments ce qui est cohérent avec le design

-bouton_1 reste haut pendant la phase "vert" du test 1, puis repasse bas pour la phase "bleu" du test 2

-On observe bien la séquence attendue : led_r_o clignote au début (état ROUGE avant le premier appui), puis led_g_o prend le relais après le premier appui sur bouton_0 (transition vers VERT), puis led_b_o prend le relais après le second appui avec bouton_1 = '0' (transition vers BLEU). C'est exactement la séquence attendue donc notre top semble fonctionner.

-end_cycle_o pulse régulièrement à chaque cycle ON/OFF des LEDs.

5. Réalisez une synthèse et étudiez le rapport de synthèse, les ressources utilisées doivent correspondre à votre schéma RTL.

State	New Encoding	Previous Encoding
init	00	00
rouge	01	01
bleu	10	11
vert	11	10

Vivado a choisi un encodage binaire séquentiel sur 2 bits pour les 4 états (INIT, ROUGE, VERT, BLEU), ce qui est cohérent avec notre design.

```
Detailed RTL Component Info :
+---Adders :
      2 Input    28 Bit      Adders := 1
+---Registers :
      28 Bit     Registers := 1
      1 Bit      Registers := 6
+---Muxes :
      2 Input    2 Bit      Muxes := 8
      4 Input    2 Bit      Muxes := 1
      2 Input    1 Bit      Muxes := 7
      4 Input    1 Bit      Muxes := 7
```

Adders :

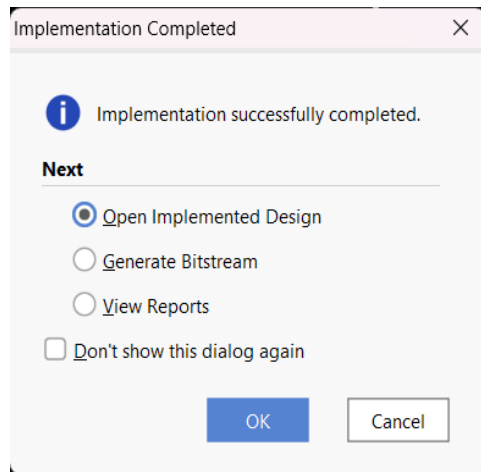
1 adder 28 bits : correspond au compteur interne de Counter_unit (l'incrémentation vers MAX_COUNT). Logique, tu n'as qu'un seul compteur de cette taille dans tout le design.

Registers :

1 register 28 bits : Le registre qui stocke la valeur courante de ce même compteur.

6 registers 1 bit : Ça colle bien avec les signaux 1 bit de led_driver si on compte: led_r_s, led_g_s, led_b_s, bouton_0_prev, update_s, end_cycle_s cela donne bien 6.

6. Effectuez le placement routage et étudiez les rapports



Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 3,688 ns	Worst Hold Slack (WHS): 0,024 ns	Worst Pulse Width Slack (WPWS): 2,750 ns
Total Negative Slack (TNS): 0,000 ns	Total Hold Slack (THS): 0,000 ns	Total Pulse Width Negative Slack (TPWS): 0,000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 98	Total Number of Endpoints: 98	Total Number of Endpoints: 66
All user specified timing constraints are met.		

Notre WNS (chemin critique) a une marge de 3,688 ns ce qui est assez confortable, cohérent avec la simplicité de notre design.

« Toutes les contraintes de temps sont respectées »

7. Générez le bitstream et vérifiez que vous avez le comportement attendu sur carte.

